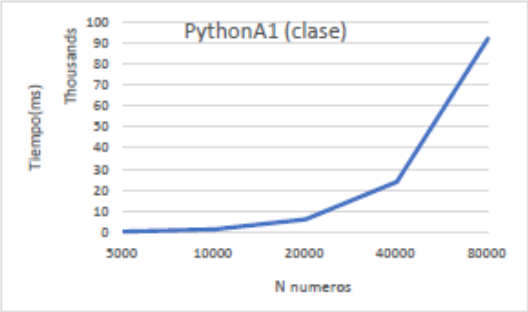
ALGORITMIA UO309022 Julio Regueira Gonzalez

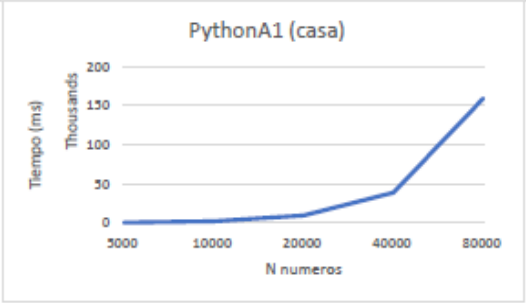
P0

(la p0 la exporté directamente de excel y da problemas para meterlo aquí así que simplemente meto fotos y ya)

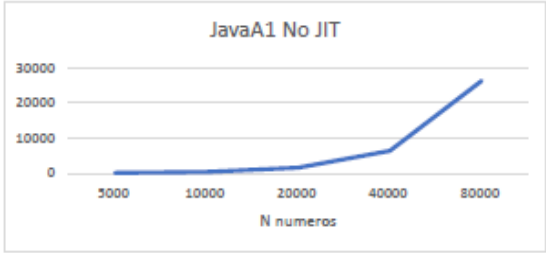
	N numeros	*Tiempo (ms)*
PythonA1 en ordenador de clase	**5000**	**349**
CPU: ???	**10000**	**1418**
	20000	**6097**
	40000	**24006**
	80000	**92037**



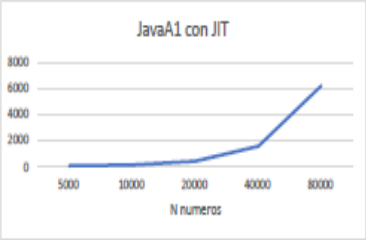
	N numeros	*Tiempo (ms)*
PythonA1 en otra maquina	**5000**	**571**
CPU: Intel Core I5 10400F 2,9GHz 12cpus	**10000**	**2338**
	20000	**9529**
	40000	**38772**
	80000	**158969**



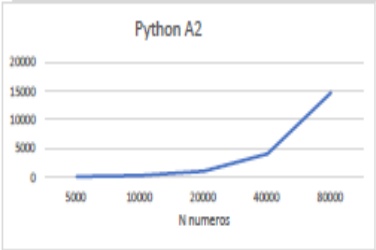
	N numeros	*Tiempo (ms)*
JavaA1 sin JIT	**5000**	**113**
CPU: Intel Core I5 10400F 2,9GHz 12cpus	**10000**	**397**
	20000	**1600**
	40000	**6340**
	80000	**26145**

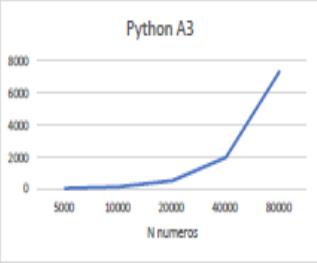


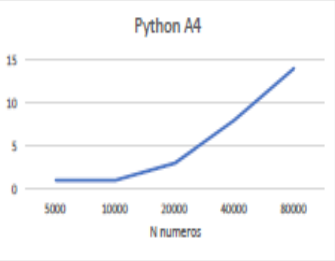
	N numeros	Tiempo (ms)
JavaA1 con JIT	5000	39
CPU: Intel Core I5 10400F 2,9GHz 12cpus	10000	100
	20000	390
	40000	1554
	80000	6213



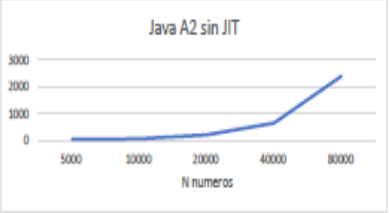
	N numeros	Tiempo A2 (ms)	Tiempo A3 (ms)	Tiempo A4 (ms)
Python 2,3 y 4	5000	77	35	1
CPU: Intel Core I5 10400F 2,9GHz 12cpus	10000	270	133	1
	20000	1025	510	3
	40000	4061	1976	8
	80000	14738	7351	14







	N numeros	Tiempo A2 NoJIT (ms)	Tiempo A2 (ms) SiJIT
Java A2 con y sin JIT	5000	25	17
CPU: Intel Core I5 10400F 2,9GHz 12cpus	10000	47	13
	20000	195	44
	40000	642	159
	80000	2407	634



	N numeros	Tiempo A3 NoJIT (ms)	Tiempo A3 (ms) SiJIT
Java A3 con y sin JIT	5000	21	15
CPU: Intel Core I5 10400F 2,9GHz 12cpus	10000	32	6
	20000	148	23
	40000	436	81
	80000	1630	301

P11

Julio Regueira Gonzalez UO309022

CPU: Intel Core i5-10400F 2.90GHz 12cpus

Memoria: 16GB DDR4

Ejercicio 1: 5,8*10^8 - 50 años

Ejercicio 2:

1 millon 10 millones 50 millones 100 millones 200 millones

Numero de valores	Tiempo ms
1 millon	5
10 millones	49
50 millones	241
100 millones	477
200 millones	957

¿Por qué a veces el tiempo medido sale 0?

Porque la función currentTimeMillis() mide el tiempo en milisegundos. Si una ejecución del algoritmo es lo suficientemente rapida (menos de 1ms) la diferencia entre el tiempo final e inicial es 0

¿A partir de qué tamaño de problema (n) empezamos a obtener tiempos fiables?

A partir de poco más de los 10 millones, pues un tiempo fiable es de 50ms

Ejercicio 3:

```
C:\Users\UO309022\Desktop\alg_JulioRegueiraGonzalezUO309022\p11>java -Xint p11.Vector3 1000000
n   tTiempo
10000   0
20000   0
40000   0
80000   1
160000  1
320000  2
640000  3
1280000 6
2560000 12
5120000 24
10240000        50
20480000        97
40960000        196
81920000        388

Fin de la medicion de tiempos *****
```

n	t
10000	0
20000	0
40000	0
80000	1
160000	1
320000	2
640000	3
1280000	6
2560000	12
5120000	24
10240000	50
20480000	97
40960000	196
81920000	388

Ejercicio 4: Vector4

Numero de repeticiones = 1

```
C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p11>java -Xint p11.Vector4 1
repeticiones = 1
n       Tiempo
10000   0
20000   0
40000   0
80000   1
160000  1
320000  2
640000  5
1280000 11
2560000 22
5120000 45
10240000        92
20480000        183
40960000        365
81920000        723

Fin de la medicion de tiempos *****
```

n	t
10000	0
20000	0
40000	0
80000	1
160000	1
320000	2
640000	5
1280000	11
2560000	22
5120000	45
10240000	92
20480000	183
40960000	365

81920000	723

Numero de repeticiones = 10

```
C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p11>java -Xint p11.Vector4
repeticiones = 10
n       Tiempo
10000   0
20000   1
40000   3
80000   8
160000  14
320000  28
640000  94
1280000 113
2560000 230
5120000 464
10240000        913
20480000        1824
40960000        3636
81920000        7278

Fin de la medicion de tiempos *****
```

n	t
10000	0
20000	1
40000	3
80000	8
160000	14
320000	28
640000	94
1280000	113
2560000	230
5120000	464
10240000	913
20480000	1824
40960000	3636
81920000	7278

Numero de repeticiones = 100

```
C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p11>java -Xint p11.Vector4 100
repeticiones = 100
n       Tiempo
10000   8
20000   18
40000   35
80000   71
160000  145
320000  282
640000  575
1280000 1140
2560000 2289
5120000 4579
10240000        9058
20480000        18315
40960000        36387
81920000        73638

Fin de la medicion de tiempos *****
```

n	t
10000	8
20000	18
40000	35
80000	71
160000	145
320000	282
640000	575
1280000	1140
2560000	2289
5120000	4579
10240000	9058
20480000	18315
40960000	36387
81920000	73638

Numero de repeticiones = 1000

```
C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p11>java -Xint p11.Vector4 1000
repeticiones = 1000
n       Tiempo
10000   88
20000   176
40000   398
80000   713
160000  1460
320000  2868
640000  5816
1280000 11597
2560000 22920
5120000 45813
^C
C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p11>
```

n	t
10000	88
20000	176
40000	398
80000	713
160000	1460
320000	2868
640000	5816
1280000	11597
2560000	22920
5120000	45813
10240000	FDT
20480000	FDT
40960000	FDT
81920000	FDT

Numero de repeticiones = 100000

```
C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p11>java -Xint p11.Vector4 100000
repeticiones = 100000
n       Tiempo
10000   8890
20000   17859
40000   36002
80000   70822
^C
C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p11>°
```

n	t
10000	8890
20000	17859
40000	36002
80000	70822
160000	FDT
320000	FDT
640000	FDT
1280000	FDT
2560000	FDT
5120000	FDT
10240000	FDT
20480000	FDT
40960000	FDT
81920000	FDT

Numero de repeticiones = 10000000

```
C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p11>java -Xint p11.Vector4 10000000
repeticiones = 10000000
n       Tiempo
^C
C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p11>
```

n	t
10000	FDT
20000	FDT
40000	FDT
80000	FDT
160000	FDT
320000	FDT
640000	FDT
1280000	FDT
2560000	FDT
5120000	FDT
10240000	FDT

20480000	FDT
40960000	FDT
81920000	FDT

¿Qué pasa con el tiempo si el tamaño del problema se multiplica por 2?

El tiempo de ejecución se multiplicaría por 2 aproximadamente de manera teorica

¿Qué pasa con el tiempo si el tamaño del problema se multiplica por otro k que no sea 2? (Pruebe, por ejemplo, para k=3 y k=4 y compruebe los tiempos obtenidos.)

Se multiplicará igual por ese mismo k

¿Razone si los tiempos obtenidos son los que se esperaban de la complejidad lineal O(n)?

Si, la relación lineal se mantiene más o menos constante para tamaños de n distintos.

Numero de repeticiones = 1

n	Tsuma	Tmaximo
10000	1	0
20000	0	0
40000	0	0
80000	1	1
160000	2	2
320000	3	3
640000	6	7
1280000	12	14
2560000	24	26
5120000	45	53
10240000	91	106
20480000	181	232
40960000	364	430
81920000	756	866

n	Tcoincidencias1	Tcoincidencias2

10000	684	0
20000	2735	1
40000	11016	2
80000	43823	1
160000	177151	2
320000	FDT	3
640000	FDT	10
1280000	FDT	13
256000	FDT	25
5120000	FDT	50
10240000	FDT	100
20480000	FDT	202
40960000	FDT	401
81920000	FDT	836

Los tiempos tienen sentido, se ve claramente que coincidencias1 tiene una complejidad cuadratica y coincidencias2 tiene una lineal. Maximo por otra parte también tiene sentido pues al ser lineal, sus tiempos son parecidos a los del algoritmo de suma

```
C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p11>java -Xint p11.Vector3 1
n   tTiempo
10000   1
20000   0
40000   0
80000   1
160000  2
320000  3
640000  6
1280000 12
2560000 24
5120000 45
10240000        91
20480000        181
40960000        364
81920000        756

Fin de la medicion de tiempos *****
```

```
C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p11>java -Xint p11.Vector5 1
repeticiones = 1
n        Tiempo
10000   0
20000   0
40000   0
80000   1
160000  2
320000  3
640000  7
1280000 14
2560000 26
5120000 53
10240000        106
20480000        232
40960000        430
81920000        866

Fin de la medicion de tiempos *****
```

```
Fin de la medicion de tiempos *****

C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p11>java -Xint p11.Vector6 1
repeticiones = 1
n        Tiempo
10000   684
20000   2735
40000   11016
80000   43823
160000  177151
^C
C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p11>
```

```
^C
C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p11>java -Xint p11.Vector7 1
repeticiones = 1
n        Tiempo
10000   0
20000   1
40000   2
80000   1
160000  2
320000  3
640000  10
1280000 13
2560000 25
5120000 50
10240000        100
20480000        202
40960000        401
81920000        836

Fin de la medicion de tiempos *****
```

P12

Julio Regueira Gonzalez UO309022

Ejercicio 1:

n	TBucle1(ms)	TBucle2(ms)	TBucle3(ms)	TBucle4(ms)
100	0	0	1	0
200	0	0	3	5
400	0	3	16	42
800	0	14	70	324
1600	0	53	266	2587
3200	1	254	1145	21023
6400	1	943	4755	167364
12800	1	4224	20214	FDT
25600	3	19301	83737	FDT
51200	8	78386	343223	FDT

```
25600   191398  10              3277056000
PS C:\Users\UO309022\Desktop\alg_JulioRegueiraGonzalezUO309022\p12> java -Xint p12.Bucle1 1
n       tiempo  repeticiones    contador
100     0       1               603
200     0       1               1340
400     0       1               2937
800     0       1               6942
1600    0       1               14938
3200    1       1               32010
6400    1       1               68272
12800   1       1               153612
25600   3       1               324273
51200   8       1               682680
102400  14      1               1501874
204800  36      1               3140282
409600  64      1               6553608
819200  136     1               13653350
PS C:\Users\UO309022\Desktop\alg_JulioRegueiraGonzalezUO309022\p12>
```

```
819200  136     1               13653350
PS C:\Users\UO309022\Desktop\alg_JulioRegueiraGonzalezUO309022\p12> java -Xint p12.Bucle2 1
n       tiempo  repeticiones    contador
100     0       1               25500
200     0       1               101000
400     3       1               482400
800     14      1               2245600
1600    53      1               8971200
3200    254     1               40985600
6400    943     1               163891200
12800   4224    1               737395200
25600   19301   1               3277056000
51200   78386   1               13107712000
PS C:\Users\UO309022\Desktop\alg_JulioRegueiraGonzalezUO309022\p12>
```

```
PS C:\Users\UO309022\Desktop\alg_JulioRegueiraGonzalezUO309022\p12> java -Xint p12.Bucle3 1
n       tiempo  repeticiones    contador
100     1       1               71400
200     3       1               323200
400     16      1               1447200
800     70      1               6416000
1600    266     1               28195200
3200    1145    1               122956800
6400    4755    1               532646400
12800   20214   1               2294118400
25600   83737   1               9831168000
51200   343223  1               41944678400
PS C:\Users\UO309022\Desktop\alg_JulioRegueiraGonzalezUO309022\p12>
```

```
51200   343223  1               41944678400
PS C:\Users\UO309022\Desktop\alg_JulioRegueiraGonzalezUO309022\p12> java -Xint p12.Bucle4 1
n       tiempo  repeticiones    contador
100     0       1               171700
200     5       1               1353400
400     42      1               10746800
800     324     1               85653600
1600    2587    1               683947200
3200    21023   1               5466454400
6400    167364  1               43711148800
PS C:\Users\UO309022\Desktop\alg_JulioRegueiraGonzalezUO309022\p12>
```

n	TBucle5(ms)	TBucle6(ms)	TBucle7(ms)
100	4	61	480
200	24	527	7188
400	114	4529	114076
800	534	39203	FDT
1600	2486	337773	FDT
3200	11485	FDT	FDT
6400	53390	FDT	FDT

```
PS C:\Users\UO309022\Desktop\alg_JulioRegueiraGonzalezUO309022\p12> javac -d . *.java
PS C:\Users\UO309022\Desktop\alg_JulioRegueiraGonzalezUO309022\p12> java -Xint p12.Bucle5 1
n       tiempo  repeticiones    contador
100     4       1               490000
200     24      1               2560000
400     114     1               12960000
800     534     1               64000000
1600    2486    1               309760000
3200    11485   1               1474560000
6400    53390   1               6922240000
PS C:\Users\UO309022\Desktop\alg_JulioRegueiraGonzalezUO309022\p12>
```

```
6400    53390   1               6922240000
PS C:\Users\UO309022\Desktop\alg_JulioRegueiraGonzalezUO309022\p12> java -Xint p12.Bucle6 1
n       tiempo  repeticiones    contador
100     61      1               7000000
200     527     1               64000000
400     4529    1               576000000
800     39203   1               5120000000
1600    337773  1               45056000000
PS C:\Users\UO309022\Desktop\alg_JulioRegueiraGonzalezUO309022\p12>
```

```
1600    337773  1               45056000000
PS C:\Users\UO309022\Desktop\alg_JulioRegueiraGonzalezUO309022\p12> java -Xint p12.Bucle7 1
n       tiempo  repeticiones    contador
100     480     1               100000000
200     7188    1               1600000000
400     114076  1               25600000000
PS C:\Users\UO309022\Desktop\alg_JulioRegueiraGonzalezUO309022\p12>
```

n	TBucle1(ms)	TBucle2(ms)	t1/t2
100	0	0	0
200	0	0	0
400	0	3	0
800	0	14	0
1600	0	53	0
3200	1	254	0,003937008
6400	1	943	0,001060445
12800	1	4224	0,000236742
25600	3	19301	0,000155432
51200	8	78386	0,000102059

n	TBucle3(ms)	TBucle2(ms)	t3/t2
100	1	0	1
200	3	0	3
400	16	3	5,333333333
800	70	14	5
1600	266	53	5,018867925
3200	1145	254	4,507874016
6400	4755	943	5,042417815
12800	20214	4224	4,785511364
25600	83737	19301	4,338479872
51200	343223	78386	4,378626285

n	Bucle4 Python (t)	Bucle4 Java NO JIT (t)	tPython/tJava
100	4	0	4
200	41	5	8,2
400	316	42	7,523809524
800	2641	324	8,151234568
1600	22371	2587	8,64746811
3200	203529	21023	9,681253865
6400	FDT	167364	-

P2

Julio Regueira Gonzalez UO309022

Burbuja

n	T ordenado	T inverso	T aleatorio
10000	418	1091	859
20000	1709	4472	3487
40000	6749	17868	13962
80000	26930	69947	55493
160000	108473	278893	225794
320000	FDT	FDT	FDT
640000	FDT	FDT	FDT
1280000	FDT	FDT	FDT

Los tiempos concuerdan con la complejidad teórica $O(n^2)$ del algoritmo. Esto se demuestra en los resultados: al duplicarse el tamaño n, el tiempo de ejecución siempre se cuadruplica aproximadamente.

Selección

n	T ordenado	T inverso	T aleatorio
10000	430	359	414
20000	1599	1433	1601
40000	6415	5588	6294
80000	25570	22311	25260
160000	101870	88730	100201
320000	FDT	FDT	FDT
640000	FDT	FDT	FDT
1280000	FDT	FDT	FDT

Es similar al anterior. Cuadra los tiempos con la complejidad O(n^2) del algoritmo. Resulta algo más rápido que burbuja, aunque no excesivamente y se puede ver en que el tiempo de ejecución se cuadriplica aproximadamente cada vez que se duplica n.

Insercion

n	T ordenado	T inverso	T aleatorio
10000	1	680	343
20000	1	2698	1352
40000	1	10791	5384
80000	2	43069	21639
160000	3	172339	86406
320000	7	FDT	FDT
640000	13	FDT	FDT
1280000	24	FDT	FDT

Los tiempos concuerdan, el caso ordenado refleja una complejidad lineal O(n) porque al duplicar n, el tiempo simplemente se duplica de forma aproximada.

En cambio, los casos inverso y aleatorio evidencian la complejidad cuadrática O(n^2) esperada, ya que sus tiempos se cuadruplican al multiplicar el tamaño por dos.

Esta diferencia ocurre porque el vector ordenado no entra al bucle while interno, mientras que los otros dos casos sí tienen que ejecutarlo para ir desplazando elementos.

Rapido

n	T ordenado	T inverso	T aleatorio
10000	3	2	4
20000	5	6	7
40000	11	13	18
80000	23	27	35
160000	48	53	73
320000	101	111	153
640000	209	231	335
1280000	440	482	675

Los tiempos reflejan la complejidad O(nlog(n)) esperada en los tres supuestos, ya que al duplicar n, el tiempo aumenta un poco más del doble.

Proyecte, a partir de las complejidades y los datos de las tablas anteriores, ¿cuántos días (1 día= 861400.000 milisegundos) tardaría cada uno de los cuatro métodos (Burbuja, Selección, Inserción y Rápido) en ordenar un vector de n= 811920.000 (o sea,

*n=80000*2^4*2^6=80000*2^10), en el caso inicialmente en orden aleatorio?*

En los metodos cuadraticos, al multiplicarse el tamaño del vector por 2^10 el tiempo de ejecución se multiplica por el cuadrado de ese factor, es decir, (2^10)^2 = 2^20 = 1.048.576.

En burbuja, si para n = 80000 tardaba 55.493ms, ahora tardará 58.188.728.928 ms, unos 673,48 dias.

Para selección, si para n = 80000 tardaba 25.260 ms, ahora tardará 26.487.0130.760 ms, unos 306,56 dias.

Para inserción, si para n= 80000 tardaba 21,639 ms, ahora tardará 22.690.136.064 ms, unos 262,62 dias.

En cambio, para el método rápido, O(nlog(n)), no crece de forma cuadrática, sino por el incremento relativo del termino logarítmico. Es decir, log(81.920.000)/log(80.000) = 1,614. Eso multiplicado por el 2^10 nos da 1652,7.

Si para n= 80.000 tardaba 35 ms, ahora tardará 57.844 ms, es decir, unos 0,00067 días.

Rapido vs Insercion a bajos tamaños 30000 repeticiones

n	T rapido	T insercion	T rápido/T inserción
10	36	20	1,8
15	56	29	1,93103448
20	76	43	1,76744186
25	106	85	1,24705882
30	128	103	1,24271845
35	163	156	1,04487179
40	174	189	0,92063492
45	222	220	1,00909091
50	248	279	0,88888889
55	271	343	0,79008746

60	313	420	0,7452381
65	351	488	0,7192623
70	391	528	0,7405303
75	425	648	0,6558642
80	475	720	0,65972222
85	512	808	0,63366337
90	500	864	0,5787037
95	536	981	0,54638124
100	578	1086	0,53222836

A partir de n aproximadante igual a 45 el algoritmo de inserción pasa a ser más lento que el rapido.

Rapido O Insercion Tiempos. A partir de n >= 42 se usa Insercion (OPCIONAL)

n	T rápido/inserción	T rapido
10	22	36
15	34	56
20	56	76
25	84	106
30	119	128
35	154	163
40	197	174
45	227	222
50	238	248
55	272	271
60	307	313
65	345	351
70	377	391
75	395	425
80	437	475
85	467	512
90	503	500
95	558	536
100	584	578

Se consiguió bajar tiempos.

P3

Julio Regueira Gonzalez UO309022

Sustraccion 1 tiene complejidad

O(1) Saltó stackoverflow a partir

de n>8000

Sustraccion 2 tiene complejidad

O(2n) Saltó stackoverflow a partir

de n>8000

Sustraccion 3 tiene complejidad O(1), (hacer regla de 3 para calcular para n=80)

Si cada vez q aumenta n una vez el tiempo aproximadamente se duplica, se tiene q duplicar desde 20 unas 60 veces. Según la complejidad O(2^(80-20), luego, 2^60. Si multiplicamos 42ms
* 2^60 = 48422703193487572992ms, unos 1.535.473.845 años aproximadamente.

```
n=20**TIEMPO=42**nVeces=1
n=21**TIEMPO=82**nVeces=1
n=22**TIEMPO=168**nVeces=1
n=23**TIEMPO=331**nVeces=1
n=24**TIEMPO=652**nVeces=1
n=25**TIEMPO=1325**nVeces=1
n=26**TIEMPO=2687**nVeces=1
n=27**TIEMPO=5295**nVeces=1
n=28**TIEMPO=10740**nVeces=1
n=29**TIEMPO=21264**nVeces=1
n=30**TIEMPO=42462**nVeces=1
n=31**TIEMPO=85174**nVeces=1
n=32**TIEMPO=170598**nVeces=1
n=33**TIEMPO=339668**nVeces=1
^C
C:\Users\UO309022\Desktop\alg_JulioRegueiraGonzalezUO309
```

Sustraccion4:

```
public static void rec4 (int n)
{
    if (n<=0)
        cont++;
    else
    {
        for (int i=0;i<n;i++){
            for (int j=0;j<n;j++){
                cont++;
            }
        }
        rec4 (n-1);
    }
}
```

n	T(ms)
100	3
200	18
400	142
800	1151
1600	9138
3200	73547

Sustraccion 5:

```
public static void rec5 (int n)
{
    if (n<=0)
        cont++;
    else
    {
        cont++;
        rec5 (n-2);
        rec5 (n-2);
        rec5 (n-2);

    }
}
```

n	T(ms)
30	397
32	1194
34	3578
36	10705
38	32166
40	97010
42	FDT

Para n= 80?

Si la complejidad del algoritmo era O(3^(n/2)). Tomando como referencia n = 30, 397ms, Habría que multiplicar ese tiempo por un factor de conversión según la complejidad

O(3^((80-30)/2)). Por ende, 397 ms * (3^(50/2)) = 3,36 * 10^14 ms, que son unos 10654,49 años.

Division 1 tiene complejidad O(n)

Division 2 tiene complejidad

O(nlogn) Division 3 tiene

complejidad O(n)

Division4:

```java
public static void rec4 (int n)
{
    if (n<=0)
        cont++;
    else
    {
        for (int i=1;i<n;i++){
            for(int j=1; j<n; j++){
                cont ++;
            }
        }
        rec4 (n/2);
    }

}
```

n	T(ms)
1000	9
2000	36
4000	137
8000	552
16000	2264
32000	9083
64000	36282
128000	FDT

Division5:

```java
public static void rec5 (int n)
{
    if (n<=0)
        cont++;
    else
    {
        cont++;
        rec5 (n/2);
        rec5 (n/2);
        rec5 (n/2);
        rec5 (n/2);
    }
}
```

n	T(ms)

1000	25
2000	104
4000	407
8000	1641
16000	6571
32000	26164
64000	104291
128000	FDT

Vector Suma y Fibonacci

```
C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p3\p3>javac -d . *.java

C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p3\p3>java -Xint p3.VectorSuma
METODO1: SUMA= -687
METODO2: SUMA= -687
METODO3: SUMA= -687
METODO1 >>> n= 100  **  TIEMPO= 897 NANOSEGUNDOS
METODO2 >>> n= 100  **  TIEMPO= 2152 NANOSEGUNDOS
METODO3 >>> n= 100  **  TIEMPO= 4450 NANOSEGUNDOS

C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p3\p3>java -Xint p3.Fibonacci
METODO1: para n= 40 es= 102334155
METODO2: para n= 40 es= 102334155
METODO3: para n= 40 es= 102334155
METODO4: para n= 40 es= 102334155
METODO1 >>> n= 40**TIEMPO= 379 NANOSEGUNDOS
METODO2 >>> n= 40**TIEMPO= 566 NANOSEGUNDOS
METODO3 >>> n= 40**TIEMPO= 1003 NANOSEGUNDOS
METODO4 >>> n= 40**TIEMPO= 6111 MILISEGUNDOS
```

Para vectorSuma, todos los metodos tienen complejidad O(n). Pese a que teóricamente los 3 escalan de la misma forma linealmente, el primero va más rápido al ser un simple bucle for, el segundo sobrecarga la pila en cada paso y por eso es más lento y el tercero, pese a dividir el problema, hace dos llamadas recursivas por cada división.

Para Fibonacci los tres primeros métodos son O(n) y el cuarto es O(1,6^n). Tienen sentido los tiempos al ser el método 1, el iterativo, el más rapido al evitar tener que gestionar la pila de llamadas recursivas del 3 o el asignar memoria a los vectores del 2. El método 4 por otra parte es el más lento al recalcular los mismos problemas una y otra vez.

PuntosTrivial

```
               14 archivos        460.667 bytes
                3 dirs  85.573.332.992 bytes libres

C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p3\p3\p3p>java p3p.PuntosTrivial datos32.txt
PUNTOS MAS CERCANOS: [27,278446, 90,131051] [27,536642, 89,236468]
SU DISTANCIA MINIMA ES= 0,931098
```

```
C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p3\p3\p3p>java p3p.PuntosTrivialTiempos 1
PUNTOS MAS CERCANOS: [68,494595, 48,860305] [68,481788, 48,872362]
SU DISTANCIA MINIMA ES= 0,017589
n=1024  Tiempo=29       Repeticiones=1
PUNTOS MAS CERCANOS: [84,274072, 35,816071] [84,299794, 35,794706]
SU DISTANCIA MINIMA ES= 0,033438
n=2048  Tiempo=16       Repeticiones=1
PUNTOS MAS CERCANOS: [62,322557, 65,949543] [62,313757, 65,955114]
SU DISTANCIA MINIMA ES= 0,010415
n=4096  Tiempo=20       Repeticiones=1
PUNTOS MAS CERCANOS: [11,705419, 6,174257] [11,710374, 6,152190]
SU DISTANCIA MINIMA ES= 0,022616
n=8192  Tiempo=77       Repeticiones=1
PUNTOS MAS CERCANOS: [25,590521, 83,416468] [25,593098, 83,421135]
SU DISTANCIA MINIMA ES= 0,005331
n=16384 Tiempo=306      Repeticiones=1
PUNTOS MAS CERCANOS: [90,802145, 7,732184] [90,803915, 7,733626]
SU DISTANCIA MINIMA ES= 0,002283
n=32768 Tiempo=1301     Repeticiones=1
PUNTOS MAS CERCANOS: [50,880788, 50,080815] [50,879856, 50,080577]
SU DISTANCIA MINIMA ES= 0,000962
n=65536 Tiempo=3570     Repeticiones=1
PUNTOS MAS CERCANOS: [32,564888, 83,471439] [32,564587, 83,471601]
SU DISTANCIA MINIMA ES= 0,000342
n=131072        Tiempo=14499    Repeticiones=1
PUNTOS MAS CERCANOS: [2,243394, 72,012711] [2,243499, 72,012827]
SU DISTANCIA MINIMA ES= 0,000156
n=262144        Tiempo=65714    Repeticiones=1
^C
C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p3\p3\p3p>
```

n	T(ms)
1024	29
2048	16
4096	20
8192	77
16384	306
32768	1301
65536	3570
131072	14499
262144	65714

Los tiempos obtenidos se adecuan a la complejidad O(n^2) del algoritmo.

P3P

PuntosDyV

```
SU DISTANCIA MINIMA ES= 0,000156
n=262144        Tiempo=65714    Repeticiones=1
^C
C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p3\p3\p3p>java p3p.PuntosDyV datos32.txt
PUNTOS MAS CERCANOS: [27,536642, 89,236468] [27,278446, 90,131051]
SU DISTANCIA MINIMA ES= 0,931098

C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p3\p3\p3p>
```

```
        at p3p.PuntosDyV.main(PuntosDyV.java:19)

C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p3\p3\p3p>java p3p.PuntosDyVTiemp
os 1
n               tiempo
-------------------------
1024            13
2048            2
4096            4
8192            7
16384           28
32768           57
65536           107
131072          192
262144          186
524288          424
1048576         926
2097152         2047
4194304         4625
8388608         10256
16777216                23362
33554432                51015
^C
C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p3\p3\p3p>
```

n	T(ms)
1024	13
2048	2
4096	4
8192	7
16384	28
32768	57
65536	107
131072	192
262144	186
524288	424
1048576	926
2097152	2047
4194304	4625
8388608	10256
16777216	23362
33554432	51015
67108864	FDT

Mi algoritmo tiene complejidad O(nlogn). En la tabla se observa que al duplicar el número de puntos n, el tiempo de ejecución aumenta ligeramente más del doble.

Devorador Java:

(cada uno de los n son el número de nodos del json)

La complejidad es O(n). No es O(n^2) porque el numero máximo de conexiones es de 8. En el supuesto de que no tuviese limite, todos serían vecinos de todos y sí sería O(n^2). Por tanto, la complejidad máxima teórica es de O(8*n). Viendo los resultados, se puede probar que si escala bien con esa complejidad. A partir de n=1024, donde los tiempos son más realistas, se ve que cada vez que n se duplica, el tiempo se duplica normalmente + un poco más.

```
C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p4\ProyectoColorear>java -cp bin;lib\json-simple-1.
1.1.jar algstudent.s4.DevoradorTiempos 100
n=4     Tiempo=3        Repeticiones=100
n=8     Tiempo=2        Repeticiones=100
n=16    Tiempo=4        Repeticiones=100
n=32    Tiempo=5        Repeticiones=100
n=64    Tiempo=5        Repeticiones=100
n=100   Tiempo=7        Repeticiones=100
n=128   Tiempo=9        Repeticiones=100
n=256   Tiempo=19       Repeticiones=100
n=512   Tiempo=27       Repeticiones=100
n=1024  Tiempo=66       Repeticiones=100
n=2048  Tiempo=93       Repeticiones=100
n=4096  Tiempo=221      Repeticiones=100
n=8192  Tiempo=417      Repeticiones=100
n=16384 Tiempo=1135     Repeticiones=100
n=32768 Tiempo=2966     Repeticiones=100
n=65536 Tiempo=6487     Repeticiones=100
```

100 repeticiones

n	T(ms)
4	3
8	2
16	4
32	5
64	5
100	7
128	9
256	19
512	27
1024	66
2048	93
4096	221
8192	417
16384	1135
32768	2966
65536	6487

Devorador Python:

```
PS C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p4> py .\devorador_tiempos.py 100
n=4     Tiempo=0.00     Repeticiones=100
n=8     Tiempo=1.04     Repeticiones=100
n=16    Tiempo=1.50     Repeticiones=100
n=32    Tiempo=3.50     Repeticiones=100
n=64    Tiempo=8.51     Repeticiones=100
n=100   Tiempo=13.51    Repeticiones=100
n=128   Tiempo=18.03    Repeticiones=100
n=256   Tiempo=39.12    Repeticiones=100
n=512   Tiempo=72.50    Repeticiones=100
n=1024  Tiempo=157.69   Repeticiones=100
n=2048  Tiempo=316.80   Repeticiones=100
n=4096  Tiempo=662.86   Repeticiones=100
n=8192  Tiempo=1365.05  Repeticiones=100
n=16384 Tiempo=2885.50  Repeticiones=100
n=32768 Tiempo=6456.58  Repeticiones=100
n=65536 Tiempo=14205.18 Repeticiones=100
PS C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\p4>
```

n	T(ms)
4	0

8	1,04
16	1,5
32	3,5
64	8,51
100	13,51
128	18,03
256	39,12
512	72,50
1024	157,69
2048	316,80
4096	662,86
8192	1365,05
16384	2885,50
32768	6456,58
65536	14205,18

P5

P5 Julio Regueira Gonzalez UO309022

1-

El estado del ferry se define como una matriz booleana

dp[i[j]. I es el índice del vehículo actual que se intenta cargar

J es la longitud total ocupada actualmente en babor

Es true si es posible posicionar los primeros i vehiculos dejando j metros en babor sin que estribor supere la longitud del barco

No haría falta guardar la longitud de estribor, porque si conocemos la suma total de los vehículos (S) y los metros en babor la longitud de estribor (j) es simplemente S-j

2-

Para cada vehiculo i con longitud v, el estado dp[i [j] es cierto si el vehiculo cabe en babor o en estribor partiendo de un estado anterior valido (dp[i-1])

Los casos base son: dp[0][0] (se usan 0 metros para colocar 0

coches) Y dp[0][j] = false para todo j > 0

3-

La complejidad del algoritmo sería O(n*L), ya que n serían el número de coches y L la longitud del barco.

El código está en la carpeta

P6

P6 Algoritmia Julio Regueira Gonzalez

(AlmacenajeContenedoresTiempos está hecho con ia)

Con 1000 repeticiones:

Fichero	Tiempo(ms)
00	142
01	95
02	204
03	190
04	174
05	96
06	79
07	80
08	19010
09	FDT

La complejidad teórica del algoritmo en el peor de los casos es O(n!). Para justificar esta complejidad, podemos aparte de ya empezar sabiendo que este problema está reconocido como np-hard y que por tanto no se conocen algoritmos de tiempo polinómico que garanticen la solución, podemos ver que el algoritmo creado genera un árbol de exploración. El primer objeto tiene 1 opción, abrir el contenedor 0, el segundo objeto tiene 2, meterlo dentro del contenedor 0 o abrir el contenedor 1 y así sucesivamente. En el peor caso, el numero de casos posibles se multiplica en cada nivel, lo que resulta en n! Combinaciones.

La parte del if(numContenedores >= mejorNumContenedores) return; se encarga de podar las ramas enteras del árbol cuando detecta que por ese camino no va a mejorar la solución, pero aun así, eso no cambia la complejidad matemática del algoritmo en el peor caso.

P6 EXTRA

Tiempos de NReinas

```
C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022\P6-Extra>java NReinasTiempos.java 1
N               Tiempo(ms)      Repeticiones    Soluciones
----------------------------------------------------------------
4               0               1               2
5               0               1               10
6               0               1               4
7               0               1               40
8               1               1               92
9               3               1               352
10              3               1               724
11              18              1               2680
12              94              1               14200
13              461             1               73712
```

P7

Tiempos de ramifica y poda

```
C:\Users\julio\OneDrive\Escritorio\alg_JulioRegueiraGonzalezUO309022>java p7.AlmacenajeContenedoresRyPTiempos 1000
Fichero         Tiempo(ms)      Repeticiones    Llamadas
----------------------------------------------------------------
test00.txt      284             1000            11
test01.txt      259             1000            14
test02.txt      218             1000            23
test03.txt      196             1000            18
test04.txt      194             1000            32
test05.txt      189             1000            43
test06.txt      186             1000            15
test07.txt      185             1000            18
test08.txt      192             1000            74
test09.txt      212             1000            160
```

El heuristico calcula una cota inferior (*lower bound*) del número mínimo de contenedores adicionales que van a hacer falta como mínimo para meter todos los objetos que quedan por asignar.
Si numContenedores (los ya usados) + lowerBound (los que harán falta como mínimo) ya supera la mejor solución conocida, mejorNumContenedores no tiene sentido seguir explorando esa rama porque nunca podrá mejorar el resultado actual, y se descarta.

Fichero	Tiempo(ms)	Reps	Llamadas
00	284	1000	11
01	259	1000	14
02	218	1000	23
03	196	1000	18
04	194	1000	32
05	189	1000	43
06	186	1000	18
07	185	1000	18
08	192	1000	74
09	212	1000	160

Y los tiempos anteriores fueron

Con 1000 repeticiones:

Fichero	Tiempo(ms)
00	142
01	95
02	204
03	190
04	174
05	96
06	79
07	80
08	19010
09	FDT

Se puede ver que los tiempos no necesariamente mejoran en todos los casos pero permite resolver los casos 08 en un tiempo razonable y 09, que antes era irresoluble en tiempo factibles.